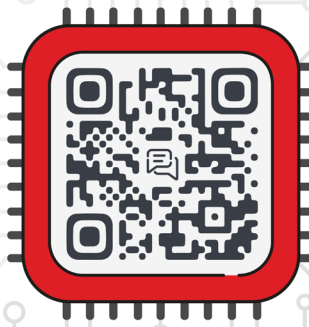




2



6



900 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

المرجعيات في الذاكرة والتغليف

د. فراس ضعيف

د. معتر النجرس



البرمجة 2

RB Informatcs; 19/04/2026

مفهوم الصنف (Class) وجدوى استخدامه في نمذجة الواقع

في لغات البرمجة التقليدية مثل C++ يتم التعامل مع أنواع بيانات أولية (Primitive Types) مثل int, double, float فعند تعريف متغير من النوع int مثلاً، يدرك المترجم (Compiler) أنه يجب حجز مساحة محددة في الذاكرة لتخزين قيمة عددية صحيحة، وتبلغ هذه المساحة عادةً 4 بايتات (Bytes). بعد ذلك يمكن إجراء العمليات الحسابية المختلفة على هذه القيمة أو تعديلها عند الحاجة.

إلا أن هذه الأنواع البسيطة لا تكفي لتمثيل الكيانات الموجودة في الواقع. فعند الرغبة في تمثيل طالب أو سيارة أو موظف داخل البرنامج، تصبح الحاجة قائمة إلى بنية أكثر تنظيماً تستطيع جمع البيانات والعمليات المرتبطة بهذا الكيان ضمن وحدة واحدة.

لهذا السبب ظهرت الأصناف (Classes)، حيث يمثل الصنف قالباً أو مخططاً يحدد الخصائص والسلوكيات الخاصة بكيان معين، بينما تمثل الكائنات (Objects) النسخ الفعلية المنشأة من هذا القالب أثناء تنفيذ البرنامج. يوضح المثال التالي تعريف صنف يمثل طالباً جامعياً:

في هذا المثال نفترض أن النظام يقبل الطلاب الجامعيين بعمر 18 سنة وأكثر

```
class Student {
    private String name;
    private int age;
    public void setAge(int a) {
        if (a >= 18) {
            age = a;
        }
    }
    public int getAge() {
        return age;
    }
}
```



المتغيرات المرجعية (References) وآلية إدارة الذاكرة

من المفاهيم الأساسية في Java فهم طبيعة المتغيرات التي تمثل الأصناف. فعند التعامل مع كائنات الأصناف، لا يخزن المتغير الكائن نفسه بشكل مباشر، وإنما يخزن مرجعاً (Reference) يسمح بالوصول إلى ذلك الكائن في الذاكرة.

مفهوم المرجع (Reference)

المرجع هو متغير يسمح بالوصول إلى كائن موجود في الذاكرة دون أن يخزن الكائن داخله. ويتحدد نوع المرجع وفق نوع الكائن الذي يمكنه الإشارة إليه.

عند كتابة: `Student s` يتم إنشاء متغير مرجعي اسمه `s` قادر على الإشارة إلى كائن من النوع `Student` إذا كان `s` متغيراً عضواً داخل صنف تكون قيمته الافتراضية `null` أما إذا كان متغيراً محلياً داخل دالة فيجب تهيئته قبل استخدامه

أما عند كتابة: `s = new Student()` فتقوم الكلمة المفتاحية `new` بإنشاء كائن جديد داخل الذاكرة المخصصة للكائنات (Heap)، ثم تجعل المرجع `s` يشير إلى هذا الكائن. وبذلك يصبح المتغير `s` مشيراً إلى الكائن الجديد الذي تم إنشاؤه.

تتبع حركة المراجع في الذاكرة

1. الحالة الأولى: نسخ المرجع (Reference Copying)

إذا تم تعريف متغير جديد وإسناد المرجع الموجود في `s` إليه: `Student t = s` فإن البرنامج لا ينشئ كائناً جديداً في الذاكرة، وإنما ينسخ المرجع الموجود في `s` إلى المتغير `t`. نتيجة لذلك، يصبح كل من `s` و `t` يشيران إلى الكائن نفسه الموجود في الذاكرة. فإذا تم تنفيذ التعليمة التالية: `s.name = "Ali"`

ثم تمت طباعة الاسم باستخدام المرجع الآخر: `System.out.println(t.name);` فستكون النتيجة الظاهرة `Ali` وذلك لأن كلا المرجعين يتعاملان مع الكائن نفسه، وبالتالي فإن أي تعديل يتم من خلال أحدهما يظهر مباشرة عند استخدام الآخر.

2. الحالة الثانية: إعادة التخصيص

إذا تمت إعادة إسناد المتغير `s` إلى كائن جديد: `s = new Student()` فإن المرجع `s` سيشير إلى الكائن الجديد الذي تم إنشاؤه، بينما يبقى المرجع `t` مشيراً إلى الكائن القديم. وبالتالي يصبح كل متغير منهما يشير إلى كائن مختلف في الذاكرة.



لفهم طريقة المقارنة بين الكائنات في Java ، سنستخدم صنفاً مبسطاً يمثل الطالب، وذلك بهدف توضيح فكرة المراجع والمقارنة بين الكائنات

```
class Student {
    String name;
    int age;
}
```

في المثال الأول

```
public class Main {
    public static void main(String[] args) {
        Student a = new Student();
        Student b = a;
        a.name = "Ali";
        a.age = 22;
        System.out.println(a == b);    // true
    }
}
```

عند تنفيذ السطر `Student b = a`

لا يتم إنشاء كائن جديد في الذاكرة، وإنما يتم نسخ المرجع الموجود في `a` إلى المتغير `b`. لذلك يصبح كل من `a` و `b` يشيران إلى الكائن نفسه.

وعند اختبار `a == b` تكون النتيجة `true` لأن المقارنة هنا تتم بين المرجعين، وكلاهما يشير إلى الكائن نفسه، لذلك فإن أي تعديل يتم من خلال أحدهما يظهر مباشرة عن استخدام الآخر.
أما في المثال الثاني:

```
public class Main {
    public static void main(String[] args) {
        Student a = new Student();
        Student b = new Student();
        a.name = "Ali";
        a.age = 22;
        b.name = a.name;
        b.age = a.age;
        System.out.println(a == b);    // false
    }
}
```

في هذا المثال تم إنشاء كائنين مستقلين باستخدام `new`. وعلى الرغم من أن القيم المخزنة داخل الكائنين أصبحت متطابقة، فإن التعبير: `a == b` يعطي النتيجة `false`، لأن المقارنة تتم بين المراجع، وليس بين محتوى الكائنات. كما أن أي تعديل لاحق على بيانات `a` لن يؤثر على بيانات `b`، والعكس صحيح.

الخلاصة: عند استخدام `==` مع الكائنات في Java، تتم مقارنة المراجع. ولا تكون النتيجة `true` إلا إذا كان المتغيران يشيران إلى الكائن نفسه في الذاكرة

التغليف (Encapsulation)

المشكلة البرمجية: عند السماح بالوصول المباشر إلى خصائص الكائن من خارج الصنف، يصبح بالإمكان إسناد قيم غير منطقية إلى البيانات المخزنة داخله. فعلى سبيل المثال عند إسناد قيمة سالبة للعمر `a.age = -100` فلن يؤدي هذا السطر إلى ظهور خطأ أثناء الترجمة، لأن القيمة ما تزال من النوع `int`، إلا أن القيمة المخزنة لا تمثل عمراً صحيحاً منطقياً.

ويكون الحل باستخدام التغليف (Encapsulation) يهدف التغليف إلى حماية البيانات الداخلية للصنف ومنع التعديل المباشر عليها من خارج الصنف، وذلك من خلال إخفاء هذه البيانات وتوفير وسائل محددة للتحكم بعملية الوصول إليها.

يتحقق ذلك عادة عبر خطوتين أساسيتين:

1. جعل المتغيرات خاصة باستخدام `private`.
2. توفير دوال عامة للتعامل مع هذه المتغيرات تعرف باسم `Getters` و `Setters`.

آلية عمل `Setters` و `Getters`

عند تعريف المتغير `age` ضمن صنف ما بالشكل التالي:

```
private int age;
```

فإن الوصول المباشر إليه من خارج الصنف يصبح غير ممكناً فعند محاولة تنفيذ:

```
System.out.print(a.age);
```

سيظهر خطأ ترجمة لأن المتغير مخفي عن الأصناف الخارجية.

لذلك يتم استخدام دالة `Setter` لإسناد القيمة بعد التحقق من صحتها، كما يتم استخدام دالة `Getter` لإرجاع القيمة المخزنة عند الحاجة إليها. كما في المثال التالي:

```
class Student {
    private int age;
    public void setAge(int a) {
        if (a >= 18)
            age = a;
    }
    public int getAge() {
        return age;
    }
}

public class Main {
    public static void main(String[] args) {
        Student a = new Student();
        a.setAge(20);
        System.out.print(a.getAge());
    }
}
```

في المثال السابق تم استخدام الدالة: `setAge()` للتحقق من أن العمر أكبر أو يساوي 18 قبل تخزينه.
أما الدالة: `getAge()` فمهمتها إرجاع القيمة المخزنة باستخدام التعليمة: `return age;`

محددات الوصول (Access Modifiers)

تستخدم محددات الوصول لتحديد نطاق رؤية الخصائص والدوال داخل البرنامج، وتوفر مستويات مختلفة من الحماية وإمكانية الوصول.

نطاق الرؤية والصلاحيات	Access Modifier
أكثر المحددات تقييداً، ولا يمكن الوصول إلى العنصر إلا من داخل الصنف نفسه	Private
لا تكتب كلمة مفتاحية، ويسمح بالوصول إلى العنصر من الأصناف الموجودة ضمن الحزمة (Package) نفسها فقط	Default
يمكن الوصول إلى العنصر من داخل الصنف نفسه، ومن الأصناف الموجودة ضمن نفس الحزمة، ومن الأصناف المشتقة منه حتى لو كانت في حزمة مختلفة.	Protected
يتيح الوصول إلى العنصر من أي مكان يمكنه رؤية الصنف	Public

تعريف الحزمة (Package)

الحزمة هي وسيلة تنظيمية تستخدم لتجميع الأصناف ذات الوظائف المتقاربة ضمن مجموعة واحدة، وغالباً ما تمثل مجلدًا يحتوي على ملفات المشروع المرتبطة بموضوع أو وظيفة معينة.

الأعضاء الساكنة (Static Members / Class Members)

في الوضع الطبيعي، تنتمي خصائص الصنف إلى الكائنات المنشأة منه. فكل كائن من النوع `Student` مثلاً اسمه وعمره الخاصان به، وتكون هذه البيانات مستقلة عن بيانات بقية الكائنات.

تسمى هذه الخصائص العادية أعضاء كائنات (Instance Members)، لأن لكل كائن نسخة خاصة منها.

إلا أن بعض الحالات تتطلب وجود بيانات مشتركة بين جميع الكائنات التابعة للصنف نفسه. وتعرف هذه البيانات باسم الأعضاء الساكنة (Static Members) أو أعضاء الصنف (Class Members).

من الأمثلة على ذلك وجود متغير يمثل العدد الإجمالي للطلاب الموجودين في البرنامج. فهذه القيمة لا تخص طالباً واحداً بعينه، وإنما تعبر عن الصنف بأكمله.

إذا احتوى البرنامج على عشرة كائنات من النوع `Student` وكان عدد الطلاب الكلي هو 10، فإن جميع الكائنات يجب أن ترى القيمة نفسها عند الاستعلام عن هذا العدد.

لتحقيق ذلك تستخدم الكلمة المفتاحية `static` فعند إضافتها إلى خاصية أو دالة معينة تصبح هذه الخاصية أو الدالة تابعة للصنف نفسه بدلاً من أن تكون تابعة لكل كائن بشكل مستقل.

ويفضل الوصول إلى العضو الساكن من خلال اسم الصنف نفسه مثل `Student.count` بدلاً من الوصول إليه عبر اسم كائن معين.

فيما يلي مثال يوضح أن `count` مشترك بين جميع كائنات `Student`

```
class Student {
    String name;
    int age;
    static int count = 0;

    Student() {
        count++;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student();

        System.out.println(Student.count); // 2
    }
}
```

